

20 — flushSync: state writes are batched, DOM reads are stale

REACTIVITY DOM TIMING

Q. You set `claps = 5` and the very next line reads the DOM — why does it still show 4?

ANSWER. Because assigning state does not touch the DOM. Svelte queues the change and applies all queued updates together after your code finishes. Any DOM read between the assignment and that batch reads the previous frame. Calling `flushSync()` right after the write forces the queue to apply immediately, so the next line sees the new value.

WHY IT WORKS. The expectation feels right because JavaScript lines run in order and every `let` change you ever made was visible on the next line. But the DOM is not the variable; it is a printout of the variable, and Svelte prints on a schedule, not per keystroke. Batching is what keeps a burst of ten writes from repainting ten times.

ClapButton.svelte

```
<script>
  import { flushSync } from 'svelte';
  let claps = $state(4);
  let counterEl;

  function read() {
    claps = 5;
    flushSync(); // apply the pending DOM update now
    console.log(counterEl.textContent); // "5" -
    without flushSync: "4"
  }
</script>

<h1 bind:this={counterEl}>{claps}</h1>
<button onclick={read}>Clap and read</button>
```

DIAGRAM CONCEPT. The Press Run: writes are stories filed to the press room and printed on a schedule; `flushSync` is a special edition printed on the spot, so the next line reads fresh news instead of yesterday's paper.

THE PRESS RUN

NO FLUSHSYNC · READS THE OLD EDITION



FLUSHSYNC() · SPECIAL EDITION PRINTS NOW



- ① **Summary.** A state write joins a queue; the DOM updates when the batch prints. Reach for `flushSync()` only when the very next line must read fresh DOM — everywhere else, let the press run on schedule.